Lock-free algorithms with such loops can result in one thread being subject to *starvation*. If another thread performs operations with the "wrong" timing, the other thread might make progress while the first thread continually has to retry its operation. Data structures that avoid this problem are wait-free as well as lock-free.

### 7.1.3. Wait-free data structures

A wait-free data structure is a lock-free data structure with the additional property that every thread accessing the data structure can complete its operation within a bounded number of steps, regardless of the behavior of other threads. Algorithms that can involve an unbounded number of retries because of clashes with other threads are thus not wait-free.

Writing wait-free data structures correctly is extremely hard. In order to ensure that every thread can complete its operations within a bounded number of steps, you have to ensure that each operation can be performed in a single pass and that the steps performed by one thread don't cause an operation on another thread to fail. This can make the overall algorithms for the various operations considerably more complex.

Given how hard it is to get a lock-free or wait-free data structure right, you need some pretty good reasons to write one; you need to be sure that the benefit outweighs the cost. Let's therefore examine the points that affect the balance.

### 7.1.4. The pros and cons of lock-free data structures

When it comes down to it, the primary reason for using lock-free data structures is to enable maximum concurrency. With lock-based containers, there's always the potential for one thread to have to block and wait for another to complete its operation before the first thread can proceed; preventing concurrency through mutual exclusion is the entire purpose of a mutex lock. With a lock-free data structure, *some* thread makes progress with every step. With a wait-free data structure, every thread can make forward progress, regardless of what the other threads are doing; there's no